文件： GLM-5.3-Flash：Frontier Intelligence, Flash Cost（官方技术博客）
来源： https://z.ai/blog/glm-5.3-flash
下载/转换时间： 2026-10-21

Z.ai 官方技术博客全文转换（含 Benchmark 表和评测口径脚注）。原页面为网页，未提供官方 PDF，本文件为网页转 PDF，内容未改写。

2026-08-26 • Research

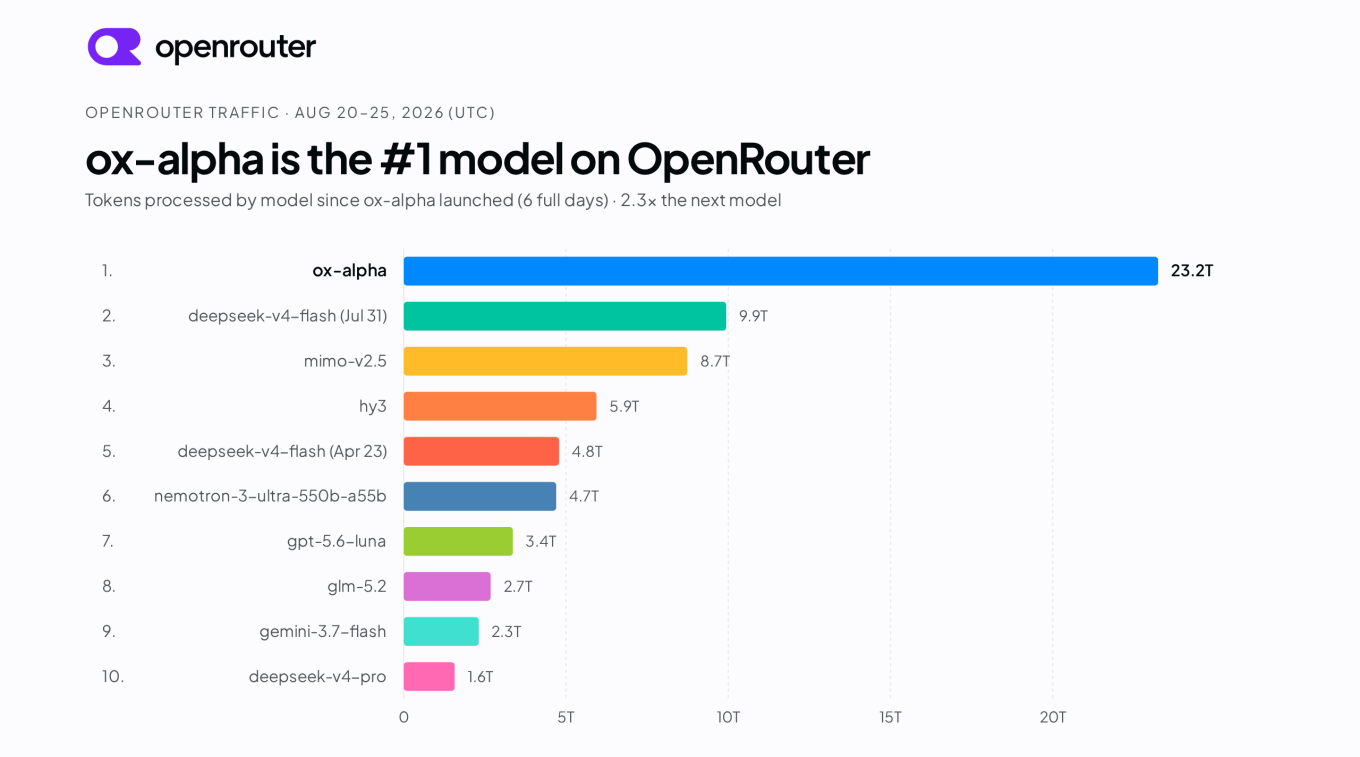
GLM-5.3-Flash: Frontier Intelligence, Flash Cost

Call it at Z.ai Z.ai Coding Plan Code with ZCode HuggingFace

We introduce GLM-5.3-Flash, the first natively multimodal model in the GLM-5 series. With 320B total parameters and just 18B active parameters, it outperforms GLM-5.2 across benchmarks and real-world workloads at one-tenth the price, while approaching Claude Opus 4.8 on coding and agentic benchmarks.

GLM-5.3-Flash starts from a newly trained base model, with its architecture and training recipe redesigned around capability and efficiency. For the first time in the GLM series, we introduce a hybrid architecture combining sparse and linear attention, sharply reducing long-context serving costs while preserving precise long-context capabilities. The model also adopts Manifold-Constrained Hyper-Connections (mHC) to further improve scaling efficiency. Together with our latest 30T-token multimodal pre-training corpus, these changes enable GLM-5.3-Flash to deliver more intelligence with less compute.

Before release, we tested GLM-5.3-Flash anonymously as `ox-alpha` on OpenCode and OpenRouter to gather user feedback. It quickly became the most popular model of the week — with all of this traffic served on Chinese AI chips.



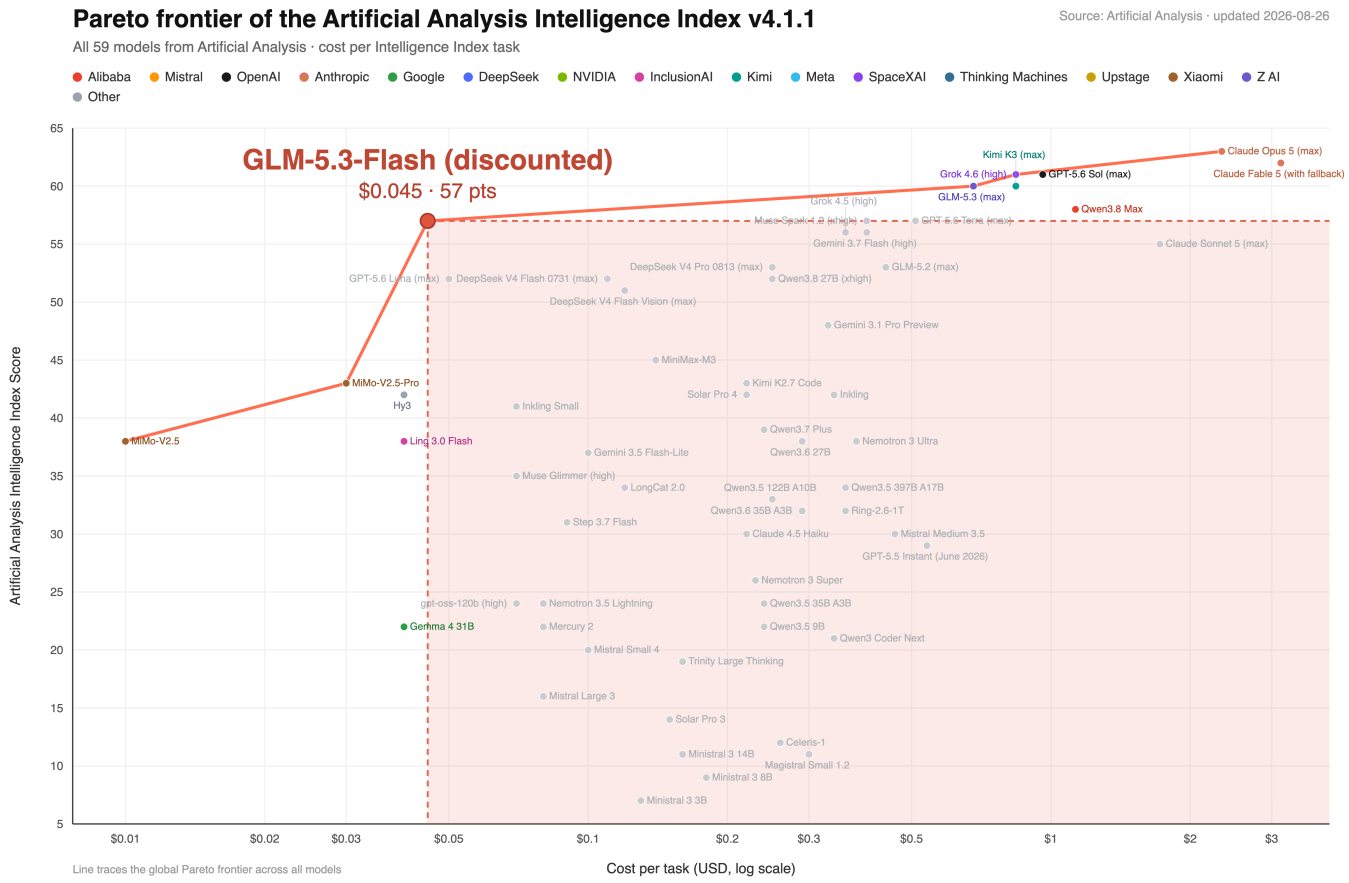
Top Models. Usage of models across OpenCode.

	7-DAY TRENDS	TOKEN USAGE
01	ox-alpha	43T
02	deepseek-v4-flash	22T
03	mimo-v2.5	11T
04	muse-spark-1.2-contributor	8.9T
05	nemotron-3-ultra	3.8T
06	hy3	2.7T
07	deepseek-v4-pro	2.0T
08	deepseek-v4-flash-vision-exp	1.8T
09	nemotron-3.5-lightning	840B
10	gpt-5.6-luna	832B

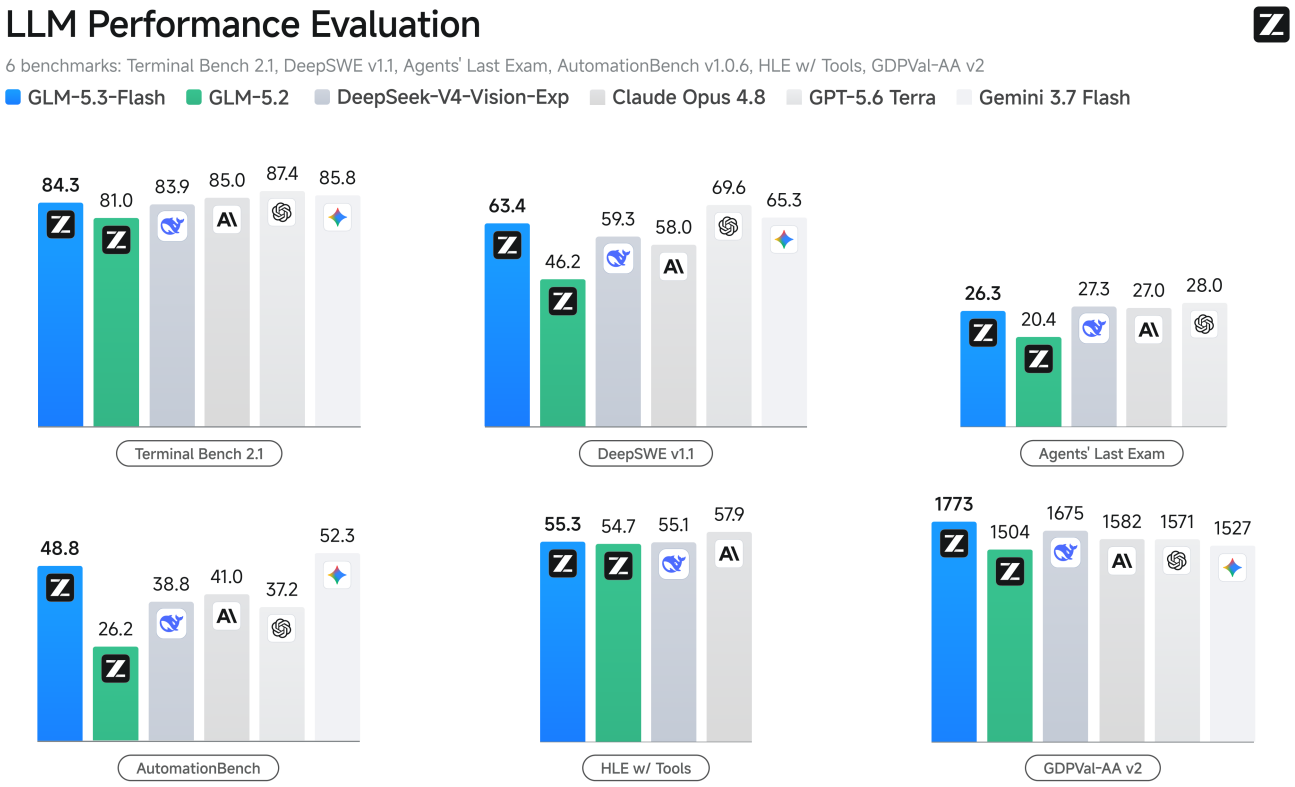
0 25% 50% 75% MAX

Competitive Performance at Flash Cost

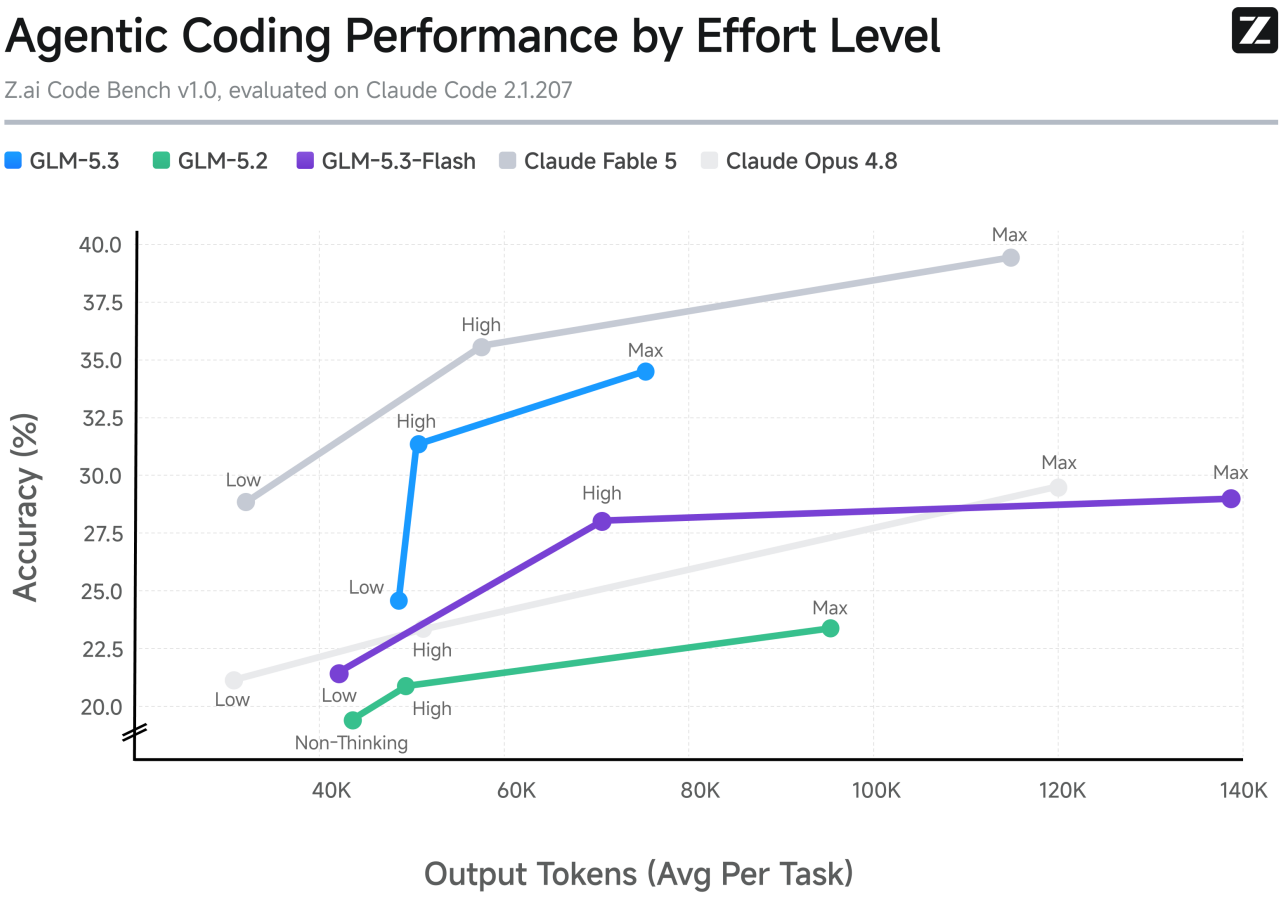
GLM-5.3-Flash pushes the Pareto frontier of the Artificial Analysis Intelligence Index v4.1.1, scoring 57 at just \$0.045 per task (discounted) — a level of intelligence previously only available at roughly 10× the cost. This makes it a highly competitive default choice for a broad range of workloads.



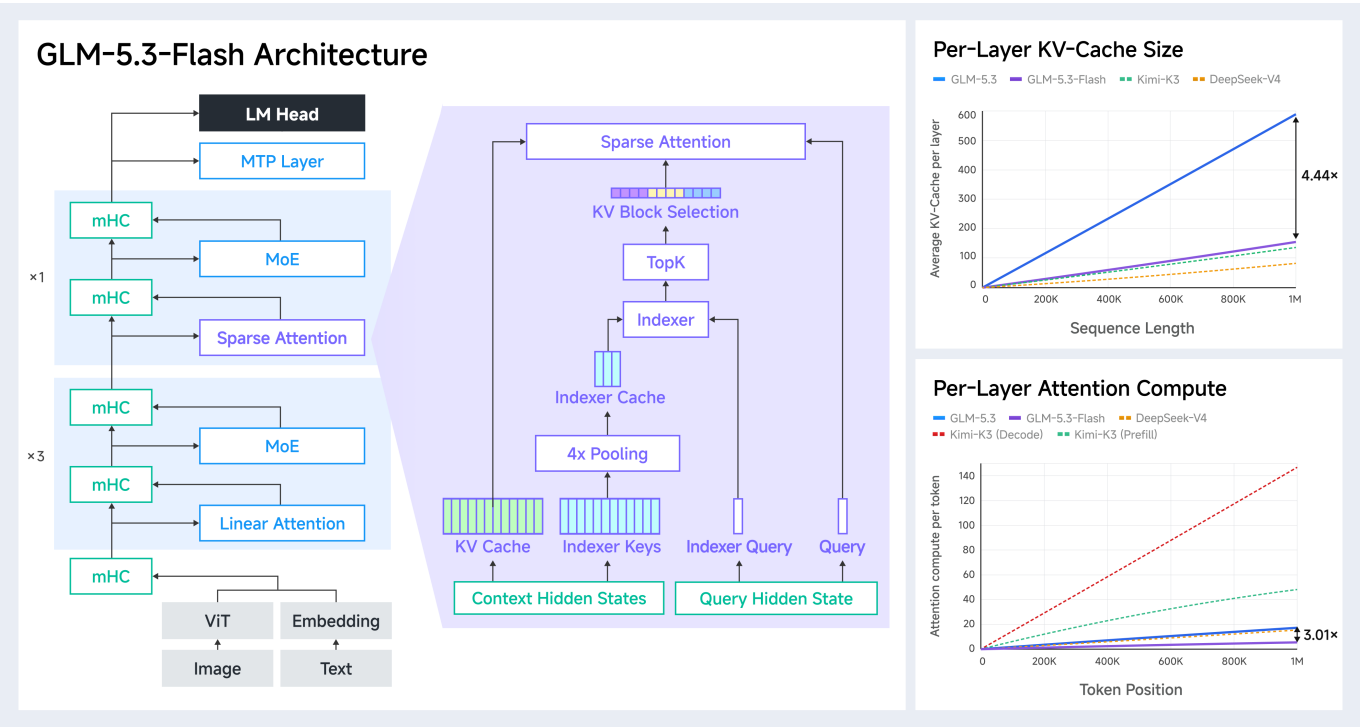
Across six coding and agentic benchmarks, GLM-5.3-Flash consistently outperforms GLM-5.2, often by a wide margin — 63.4 vs. 46.2 on DeepSWE v1.1 and 48.8 vs. 26.2 on AutomationBench — while approaching Claude Opus 4.8 overall.



This holds on our in-house coding evaluation as well: on Z.ai Code Bench v1.0, GLM-5.3-Flash clearly outperforms GLM-5.2 at every effort level, and at max effort nearly matches Claude Opus 4.8 (29.0 vs. 29.5).



Architecture for Extreme Efficiency



Compared with the GLM-4.5 series, GLM-5.3-Flash is specifically designed for ultra-low-cost inference. Despite a similar total parameter count (320B vs. 355B), it nearly halves both the activated parameter count (18B vs. 32B) and the number of layers (45 vs. 92).

To minimize attention costs in long-context scenarios, we use a hybrid architecture combining linear and sparse attention. Linear attention captures local dependencies through state modeling, while sparse attention retrieves relevant global context through a lightweight indexer. To further reduce the latency and memory overhead of the indexer at a 1M-token context length, we introduce IndexPool, which compresses four indexer key vectors into one through weighted pooling.

To illustrate the efficiency of our architecture, we compare the per-token compute and KV cache size of GLM-5.3-Flash against GLM-5.3 and two recent open models: DeepSeek-V4-Flash and Kimi-K3. For a fair comparison among different scales, we calculate the attention compute per head per layer and average KV cache size per layer (BF16). Compared with GLM-5.3, GLM-5.3-Flash uses 3.0× less attention compute and a 4.4× smaller KV cache. GLM-5.3-Flash has the lowest attention compute among all models compared. The KV cache size is still slightly larger than Kimi-K3 and DeepSeek-V4-Flash, leaving further room for improvement.

The overall architecture improvements, combined with optimized pre-training corpus, enable GLM-5.3-Flash to produce more intelligence with less compute.

The table below compares GLM-5.3-Flash-Base with our previous base models and DeepSeek-V4-Flash-Base. GLM-5.3-Flash-Base outperforms GLM-4.5-Base overall and remains competitive with GLM-5-Base across most benchmarks.

| Benchmark | GLM-4.5-Base
| GLM-5-Base
| DeepSeek-V4-Flash-Base
GLM-5.3-Flash-Base
Activated Params	32B	40B	13B	18B
Total Params	355B	744B	284B	320B
MMLU	86.1	88.3	88.5	88.1
BBH	86.2	87.4	84.9	86.6
HellaSwag	87.1	88.1	85.3	87.1

| LiveCodeBench-Base | 28.1 | 34.4 | 29.9 | 37.6 |
| SimpleQA | 30.0 | 36.0 | 31.2 | 33.5 |

(Results for DeepSeek-V4-Flash-Base were evaluated using our internal evaluation framework to control for implementation differences)

Visual Intelligence in the Coding Loop

Visual coding is not just about processing images. It expands the boundary of what coding can reach. For tasks such as frontend development, game development, and 3D simulation, the final output is not code alone, but an interface, an interaction, or a world experienced by the user. Many failures only surface through rendering, interaction, or playtesting. CUA further extends coding beyond programmable systems into visible and interactive environments. Vision therefore needs to be natively integrated into the model, enabling it to decide when to observe and use visual feedback to guide subsequent actions.

We developed data synthesis pipelines for visual coding, with a focus on visual self-evaluation and test-time improvement. The resulting trajectories require the model to interact with environments, inspect its own outputs, and refine them iteratively. For frontend coding, we also explored reinforcement learning with environment feedback and further strengthened GUI evaluation through agent-based verification grounded in real user flows. This extends validation beyond functional correctness to the rendered and interactive product.

Code lets the model build and change the world. Vision lets it enter the world people see and use. The key is to close the loop between generation and visual feedback: the model can render its output, inspect what users would actually see, identify visual issues, and revise the underlying artifact accordingly. This applies not only to interfaces and games, but also to visually structured outputs such as presentations. In the example below, GLM-5.3-Flash detects layout problems in a generated slide and refines the composition through visual self-verification.



Initial Version with Layout Issues

官方公开资料 | 2026-10-21 下载



After Visual Self-Verification

Beyond Coding — Your Partner at Work

Coding capabilities provide an important foundation for intelligent knowledge work, while visual intelligence extends these capabilities to a broader range of professional tasks. A substantial portion of professional activities involves interpreting heterogeneous visual and structured information, including documents, spreadsheets, presentations, dashboards, interfaces, and meeting artifacts.

Visual intelligence extends the model’ s capabilities beyond code-centric environments by enabling it to jointly reason over textual, visual, and structural context. Rather than requiring users to explicitly translate their working environment into textual instructions, the model can directly interpret the artifacts associated with a task and identify relevant information. It can also assess its own outputs against the visual context and intended outcome, enabling more effective self-verification and refinement — including stronger judgments of presentation quality and aesthetics.

These capabilities become particularly evident in the following examples of professional workflows.

Serving at Scale on Chinese AI Chips

Over the past week, we have served GLM-5.3-Flash on a large-scale cluster of Chinese AI chips, supported by a high-bandwidth interconnect and a serving stack optimized for the underlying hardware.

To overcome the relatively limited compute and memory capacity of individual chips, we built a dedicated inference engine for this architecture on top of SGLang. Notably, this effort was accelerated by our GLM-5.3-powered infrastructure agent, which assisted engineers in developing and optimizing kernels, diagnosing performance bottlenecks, and improving the serving stack — creating a feedback loop in which the model helped optimize the system serving the model itself.

These chips are primarily constrained by memory capacity and bandwidth, especially when supporting context lengths of up to one million tokens. This calls for aggressive memory optimization, including compute-for-bandwidth and communication-for-bandwidth techniques tailored to the underlying architecture. Our stack combines intra-node tensor parallelism for linear attention and the LM head, ReplaySSM, W8A8 quantization, hybrid INT8/FP8/BF16 cache quantization, and Layer Split.

At cluster scale, our production-grade Encode–Prefill–Decode (EPD) disaggregated architecture separates multimodal encoding, prompt prefill, and token-by-token decoding into independently scheduled and scalable worker pools, enabling efficient and reliable serving.

Compared with our initial baseline on the same hardware, we achieved a 3× improvement in end-to-end serving performance, reaching hardware efficiency and per-token cost comparable to mainstream NVIDIA GPUs. This demonstrates that Chinese chips can support frontier-model inference efficiently and economically at scale.

Conclusion

GLM-5.3-Flash shows that frontier intelligence does not have to come at frontier cost. This is not the result of any single trick, but of three layers working together: an architecture that delivers stronger capability from less compute, a richer multimodal pre-training corpus, and infrastructure co-designed with inference hardware. We are now scaling this recipe to larger models — GLM-5.3-Flash pushes the cost-performance frontier, and the lessons from building it are already shaping our next frontier model.

Getting started with GLM-5.3-Flash

We've rolled out GLM-5.3-Flash to all GLM Coding Plan users. GLM-5.3-Flash gives you **3x** the usable quota of GLM-5.3. Try **GLM-5.3-Flash** in z.ai/subscribe.

Unlock GLM-5.3-Flash's multimodal capabilities in ZCode with **Browser Use** and **Computer Use** : the agent clicks through and visually verifies web pages, and operates your desktop apps.

The model weights of GLM-5.3-Flash are publicly available on Hugging Face. For local deployment, GLM-5.3-Flash currently supports inference frameworks including SGLang, vLLM and TokenSpeed. Support for more frameworks is on the way.

| Benchmark | GLM-5.3-Flash
| GLM-5.2
| DeepSeek-V4-Vision-Exp
| Opus 4.8
| GPT-5.6 Terra
Gemini 3.7 Flash
Coding						
Terminal Bench 2.1	84.3	81.0	83.9	85.0	87.4	85.8
DeepSWE
v1.1	63.4	46.2	59.3	58.0	69.6	65.3
NL2Repo	56.3	48.9	57.7	69.7	-	-
Agentic						
Toolathlon Verified	78.4	59.9	75.9	76.2	74.9	-
AutomationBench
v1.0.6	48.8	26.2	38.8	41.0	37.2	52.3
Agents' Last Exam	26.3	20.4	27.3	27.0	28.0	-
HLE w/ Tools	55.3	54.7	55.1	57.9	-	-
GDPval-AA v2	1773	1504	1675	1582	1571	1527
Vision						
OfficeQA Pro	62.4	-	57.9	48.9	-	-
CharXiv Reasoning
w/ Tools	89.4	-	80.4	89.9	88.0	88.7
Chartography
w/ Tools	78.0	-	64.3	75.0	68.0	65.0
BabyVision	53.4	-	35.1	46.8	61.6	70.9
MVbench	77.8	-	69.4	67.1	75.0	82.2

MMVU	80.5	-	72.7	67.4	75.8	82.3
OSWorld 2.0	59.1	-	54.9	54.8	50.2	47.9
Vision2Web	77.8	-	67.6	76.1	67.5	76.5

Footnotes

- **HLE w/ tools (full set)** : We use sampling parameters of `temperature=1.0` and `top_p=0.95` for evaluation, with a maximum generation length of `163,840` tokens. The evaluation is conducted with a maximum context length of `300,000` tokens, using a context management strategy. We use GPT-5.6-luna (medium) as the judge model.
- **NL2Repo** : We evaluated NL2Repo with temperature=1.0, top_p=1.0, and max_new_tokens=64k under 1M context. To prevent hacking, we use rule-based and a LLM-based judgement to prevent malicious behaviors (e.g., unauthorized pip or curl operations).
- **DeepSWE** : We run DeepSWE using the mini-swe-agent harness with `temperature=0.95`, `top_p=1.0`, `timeout=6h` and 400K context.
- **Terminal-Bench 2.1** : We evaluate in Claude Code 2.1.207 with temperature=1.0, top_p=1, max_new_tokens=65536 with 6h timeout.
- **Agent’ s Last Exam** : We evaluate ALE using the official evaluation protocol with the Claude Code harness (reasoning effort=max, 1M context, and 64K maximum output). Tool Search is disabled, and results are scored by the official ALE evaluators.
- **Toolathlon Verified** : We obtain all results via the official evaluation service and report pass@1 averaged over 3 independent runs.
- **AutomationBench** : We evaluate on AutomationBench **v1.0.6** , incorporating the fix for the `null` -type handling issue introduced in PR #13.
- **GDPval-AA v2** : Models are evaluated by Artificial Analysis.
- **BabyVision** : We use temperature=1.0, top_p=0.95, and a maximum context length of 164K tokens. We resize the input images such that their shorter side is at least 1.5K pixels, consistent with other baselines.
- **OfficeQA Pro** : We evaluate the agent on the Treasury Bulletin PDF corpus without providing access to embedded text. We use a temperature of 1.0, top_p of 0.95, and a maximum context length of 512K tokens.
- **CharXiv Reasoning** : We use temperature=1.0, top_p=0.95, and a maximum context length of 256K tokens.
- **Chartography** : We use temperature=1.0, top_p=0.95, and a maximum context length of 256K tokens.
- **MVBench and MMVU** : We use temperature=1.0, top_p=0.95, and a maximum context length of 256K tokens. For models that natively accept video input, such as Gemini 3.7 Flash,